

SISTEMA DE GESTIÓN DE VENTAS MERCURIO

Documento de Diseño y Modelado de Datos

Base de Datos Relacional: DB_GESTIONVENTAS

Motor de DB: Microsoft SQL Server

Entorno de Desarrollo: Java 20 (JDK 20.0.2) + JavaFX

Autor: Ana Luz Teves Tessaro

Fecha: Septiembre de 2026

Versión: 1.0

1. Introducción y Contexto de la Base de Datos

El presente documento detalla el diseño relacional y el modelado físico de la base de datos **DB_GESTIONVENTAS**, el núcleo de almacenamiento estructurado del sistema de gestión de ventas **Mercurio**. La implementación física se realiza en el motor de base de datos relacional **Microsoft SQL Server**, garantizando integridad referencial, consistencia transaccional (ACID) y un rendimiento óptimo de consultas.

1.1 Entorno Técnico de Operación

La base de datos interactúa con la aplicación de escritorio construida bajo una arquitectura modular robusta, cuyas especificaciones técnicas son las siguientes:

- **Motor de Base de Datos:** Microsoft SQL Server (Instancia local o en servidor central).
- **Nombre de Base de Datos:** DB_GESTIONVENTAS
- **Usuario de Base de Datos:** analuzteves
- **Lenguaje de Programación:** Java 20 (JDK 20.0.2) de 64 bits utilizando la API JDBC para la conectividad y ejecución de sentencias SQL preparadas (PreparedStatement) para mitigar riesgos de inyección SQL.
- **Interfaz Gráfica de Usuario (GUI):** JavaFX 20, mapeando eventos directamente con controladores y capas de servicio correspondientes.
- **Patrón de Arquitectura:** MVC + Services (Modelos, Controladores, Vistas FXML y Servicios intermedios que encapsulan la lógica de persistencia e interactúan con la base de datos).

2. Arquitectura de Entidades y Relaciones (DER)

El modelado de datos de **Mercurio** se compone de 9 entidades físicas que gestionan el ciclo completo de la venta, desde el control de accesos por roles, la auditoría detallada de acciones, la trazabilidad de arqueos de caja, hasta la comercialización de productos segmentados por categoría. La vinculación de estas tablas asegura un flujo cerrado donde cada proceso está estrictamente controlado.

2.1 Tabla Resumen de Entidades

Tabla	Descripción Funcional	Cardinalidades Críticas
PERFIL	Define los perfiles de acceso autorizados del sistema: Gerente, Administrador y Vendedor.	1:N con USUARIO
USUARIO	Registra las credenciales de ingreso, datos personales y	1:N con VENTA y

	perfiles de los empleados.	AUDITORIA
AUDITORIA	Almacena la bitácora técnica de seguridad del sistema para control del Administrador.	N:1 con USUARIO
Caja	Registra los arqueos financieros, aperturas, cierres, montos iniciales y finales por día.	Independiente / Vinculado por DNI
CLIENTE	Almacena los datos personales de clientes registrados para la facturación personalizada.	1:N con VENTA
VENTA	Cabecera de transacción comercial que consolida la fecha, total de venta y el cajero/cliente.	1:N con DETALLE_VENTA; N:1 con USUARIO/CLIENTE
DETALLE_VENTA	Detalle transaccional que contiene las cantidades y precios de los productos vendidos.	N:1 con VENTA y PRODUCTO
PRODUCTO	Catálogo físico de artículos disponibles con control de stock, precio y estado.	N:1 con CATEGORIA; 1:N con DETALLE_VENTA
CATEGORIA	Segmentación del catálogo de productos gamer (ej. Periféricos, Componentes, Consolas).	1:N con PRODUCTO

3. Diccionario de Datos Detallado

A continuación se describen las especificaciones físicas detalladas para cada una de las tablas en SQL Server, incluyendo nombres de campo, tipos de datos recomendados, claves (Primaria PK / Foránea FK), nulidad y reglas lógicas de negocio asociadas.

3.1 Tabla: PERFIL

Almacena los perfiles de usuario autorizados para la segregación de funciones.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_perfil	INT IDENTITY	PK	No	Identificador autoincremental del perfil.

descripcion	VARCHAR(50)	-	No	Nombre del perfil de acceso (Vendedor, Gerente, Administrador). Único.
-------------	-------------	---	----	--

3.2 Tabla: USUARIO

Registra el personal del sistema, sus credenciales hash y estados de acceso.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_usuario	INT IDENTITY	PK	No	Identificador autoincremental de usuario.
nombreyape	VARCHAR(100)	-	No	Nombre y Apellidos completos del empleado.
DNI	VARCHAR(20)	-	No	Documento Nacional de Identidad del usuario. Único.
email	VARCHAR(100)	-	No	Correo electrónico institucional. Formato único.
id_perfil	INT	FK	No	Vínculo con PERFIL.id_perfil. Define privilegios.
estado	VARCHAR(20)	-	No	Estado actual de la cuenta: 'Activo' o 'Suspendido'.

3.3 Tabla: AUDITORIA

Bitácora automatizada que registra las modificaciones y transacciones por motivos de seguridad.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_auditoria	INT IDENTITY	PK	No	Identificador autoincremental del log.
fecha_hora	DATETIME	-	No	Fecha y hora exacta del evento. Default: GETDATE().
id_usuario	INT	FK	No	Vínculo con USUARIO.id_usuario. Responsable de la acción.
modulo	VARCHAR(50)	-	No	Módulo del sistema modificado (Clientes, Ventas, Stock).

accion	VARCHAR(250)	-	No	Detalle de la acción realizada (ej: 'Alta de cliente DNI 30193').
--------	--------------	---	----	---

3.4 Tabla: Caja

Manejo financiero de los arqueos y turnos diarios. Control de apertura y cierre.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_caja	INT IDENTITY	PK	No	Identificador único de la sesión de caja.
DNI	VARCHAR(20)	-	No	DNI del vendedor responsable que realiza la apertura/cierre.
fecha_apertura	DATETIME	-	No	Fecha y hora de inicio del arqueo diario.
fecha_cierre	DATETIME	-	Sí	Fecha y hora de finalización del arqueo. Nulo si está abierta.
monto_inicial	DECIMAL(18,2)	-	No	Dinero físico inicial declarado en caja para cambio.
monto_final_sistema	DECIMAL(18,2)	-	Sí	Dinero acumulado calculado automáticamente por ventas.
monto_final_real	DECIMAL(18,2)	-	Sí	Dinero físico real contado al cierre de caja.
diferencia	DECIMAL(18,2)	-	Sí	Diferencia (Real - Sistema). Registra sobrantes/faltantes.

3.5 Tabla: CLIENTE

Almacenamiento de información del cliente para la personalización de comprobantes.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_cliente	INT IDENTITY	PK	No	Identificador único y secuencial del cliente.

nomYape	VARCHAR(100)	-	No	Razón social, nombre y apellido del cliente.
documento	VARCHAR(20)	-	No	Documento fiscal / DNI del cliente. Valor único.
email	VARCHAR(100)	-	Sí	Correo electrónico de contacto.
telefono	VARCHAR(20)	-	Sí	Número de contacto móvil o fijo.

3.6 Tabla: VENTA

Cabecera transaccional que agrupa los pedidos comerciales concretados.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_venta	INT IDENTITY	PK	No	Identificador único de la transacción.
fecha_venta	DATETIME	-	No	Fecha y hora del procesamiento de pago. Default: GETDATE().
total_venta	DECIMAL(18,2)	-	No	Suma total acumulada de los detalles asociados. Mayor a 0.
id_usuario	INT	FK	No	Usuario (vendedor) que ejecutó y cerró la venta.
id_cliente	INT	FK	No	Vínculo con CLIENTE.id_cliente receptor de la venta.

3.7 Tabla: DETALLE_VENTA

Partidas individuales que componen cada una de las ventas de productos.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_detalleVenta	INT IDENTITY	PK	No	Identificador secuencial del detalle.
cantidad	INT	-	No	Cantidad de unidades comercializadas. Debe ser > 0.

precio_unitario	DECIMAL(18,2)	-	No	Precio histórico del producto al momento de vender.
id_venta	INT	FK	No	Vínculo a la cabecera VENTA.id_venta.
id_producto	INT	FK	No	Vínculo al artículo PRODUCTO.id_producto.

3.8 Tabla: PRODUCTO

Maestro de inventarios gamer para control de existencias físicas.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_producto	INT IDENTITY	PK	No	Identificador único del producto.
nombre	VARCHAR(100)	-	No	Denominación comercial del artículo. Único.
descripcion	VARCHAR(250)	-	Sí	Características técnicas complementarias.
precio_venta	DECIMAL(18,2)	-	No	Precio unitario para venta al público. Debe ser > 0.
stock	INT	-	No	Inventario disponible en almacén. Debe ser >= 0.
estado	VARCHAR(20)	-	No	Control de disponibilidad ('Disponible', 'Suspendido').
id_categoria	INT	FK	No	Mapeo hacia CATEGORIA.id_categoria.

3.9 Tabla: CATEGORIA

Agrupamiento lógico de los productos gamer.

Campo	Tipo de Dato	Clave	Nulo	Descripción / Reglas de Negocio
id_categoria	INT IDENTITY	PK	No	Identificador de categoría.
nombre	VARCHAR(50)	-	No	Nombre del sector comercial (ej: 'Teclados', 'Mouses').

			Único.	
descripcion	VARCHAR(250)	-	Sí	Descripción del alcance de la categoría.

4. Script de Inicialización Física (DDL SQL Server)

Se presenta el script estructurado de comandos DDL compatible con SQL Server para generar el esquema de base de datos **DB_GESTIONVENTAS** con sus respectivas relaciones referenciales, campos obligatorios, restricciones UNIQUE e índices:

```
USE master;
GO
IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'DB_GESTIONVENTAS')
BEGIN
    CREATE DATABASE DB_GESTIONVENTAS;
END
GO
USE DB_GESTIONVENTAS;
GO

-- 1. CREACIÓN DE TABLA PERFIL
CREATE TABLE PERFIL (
    id_perfil INT IDENTITY(1,1) PRIMARY KEY,
    descripcion VARCHAR(50) NOT NULL UNIQUE
);
GO

-- 2. CREACIÓN DE TABLA USUARIO
CREATE TABLE USUARIO (
    id_usuario INT IDENTITY(1,1) PRIMARY KEY,
    nombreyape VARCHAR(100) NOT NULL,
    DNI VARCHAR(20) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    id_perfil INT NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'Activo',
    CONSTRAINT FK_USUARIO_PERFIL FOREIGN KEY (id_perfil) REFERENCES PERFIL(id_perfil)
);
GO

-- 3. CREACIÓN DE TABLA AUDITORIA
CREATE TABLE AUDITORIA (
    id_auditoria INT IDENTITY(1,1) PRIMARY KEY,
    fecha_hora DATETIME NOT NULL DEFAULT GETDATE(),
    id_usuario INT NOT NULL,
    modulo VARCHAR(50) NOT NULL,
    accion VARCHAR(250) NOT NULL,
    CONSTRAINT FK_AUDITORIA_USUARIO FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario)
);
```

GO

-- 4. CREACIÓN DE TABLA CAJA

```
CREATE TABLE Caja (
    id_caja INT IDENTITY(1,1) PRIMARY KEY,
    DNI VARCHAR(20) NOT NULL,
    fecha_apertura DATETIME NOT NULL DEFAULT GETDATE(),
    fecha_cierre DATETIME NULL,
    monto_inicial DECIMAL(18,2) NOT NULL DEFAULT 0.00,
    monto_final_sistema DECIMAL(18,2) NULL,
    monto_final_real DECIMAL(18,2) NULL,
    diferencia DECIMAL(18,2) NULL
```

);

GO

-- 5. CREACIÓN DE TABLA CLIENTE

```
CREATE TABLE CLIENTE (
    id_cliente INT IDENTITY(1,1) PRIMARY KEY,
    nomYape VARCHAR(100) NOT NULL,
    documento VARCHAR(20) NOT NULL UNIQUE,
    email VARCHAR(100) NULL,
    telefono VARCHAR(20) NULL
```

);

GO

-- 6. CREACIÓN DE TABLA CATEGORIA

```
CREATE TABLE CATEGORIA (
    id_categoria INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(250) NULL
```

);

GO

-- 7. CREACIÓN DE TABLA PRODUCTO

```
CREATE TABLE PRODUCTO (
    id_producto INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    descripcion VARCHAR(250) NULL,
    precio_venta DECIMAL(18,2) NOT NULL CHECK(precio_venta > 0),
    stock INT NOT NULL CHECK(stock >= 0),
    estado VARCHAR(20) NOT NULL DEFAULT 'Disponible',
    id_categoria INT NOT NULL,
    CONSTRAINT FK_PRODUCTO_CATEGORIA FOREIGN KEY (id_categoria) REFERENCES
```

CATEGORIA(id_categoria)

);

GO

-- 8. CREACIÓN DE TABLA VENTA

```
CREATE TABLE VENTA (
    id_venta INT IDENTITY(1,1) PRIMARY KEY,
    fecha_venta DATETIME NOT NULL DEFAULT GETDATE(),
    total_venta DECIMAL(18,2) NOT NULL CHECK(total_venta >= 0),
    id_usuario INT NOT NULL,
    id_cliente INT NOT NULL,
    CONSTRAINT FK_VENTA_USUARIO FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario),
```

```

        CONSTRAINT FK_VENTA_CLIENTE FOREIGN KEY (id_cliente) REFERENCES CLIENTE(id_cliente)
    );
GO

-- 9. CREACIÓN DE TABLA DETALLE_VENTA
CREATE TABLE DETALLE_VENTA (
    id_detalleVenta INT IDENTITY(1,1) PRIMARY KEY,
    cantidad INT NOT NULL CHECK(cantidad > 0),
    precio_unitario DECIMAL(18,2) NOT NULL CHECK(precio_unitario > 0),
    id_venta INT NOT NULL,
    id_producto INT NOT NULL,
    CONSTRAINT FK_DETALLE_VENTA_VENTA FOREIGN KEY (id_venta) REFERENCES VENTA(id_venta) ON
DELETE CASCADE,
    CONSTRAINT FK_DETALLE_VENTA_PRODUCTO FOREIGN KEY (id_producto) REFERENCES
PRODUCTO(id_producto)
);
GO

```

5. Reglas de Negocio Aplicadas a la Base de Datos

La base de datos relacional no actúa únicamente como un repositorio pasivo de información, sino que trabaja en consonancia con la capa de **Services** en JavaFX para garantizar el cumplimiento de restricciones clave del negocio:

5.1 Control de Caja Abierta para Ventas

Para registrar una tupla en la tabla **VENTA**, el sistema realiza una consulta previa a la tabla **Caja** buscando si existe una fila cuyo `fecha_cierre IS NULL` para el DNI del vendedor en el día en curso. En caso negativo, la transacción es cancelada en el Service de Java disparando un objeto de excepción que interrumpe la escritura física en la base de datos.

5.2 Reducción de Stock y Excepción de Existencias

Al procesar una venta, por cada fila agregada en **DETALLE_VENTA**, se ejecuta una operación `UPDATE` sobre **PRODUCTO** reduciendo el atributo `stock`. La restricción de tabla `CHECK (stock >= 0)` configurada a nivel de motor de base de datos SQL Server garantiza que ningún proceso (concurrente o no) pueda dejar las existencias en números negativos. Si esto ocurre, SQL Server lanza inmediatamente un error `SQLException` que la aplicación Java captura en el controlador para detener el proceso y notificar al usuario de forma amigable.

5.3 Registro Activo de Auditorías

Cualquier operación de escritura (`INSERT`, `UPDATE`, `DELETE`) en tablas críticas como **PRODUCTO**, **Caja** o **CLIENTE** es interceptada en la capa Service de Java, donde se dispara una transacción combinada que

añade una entrada en la tabla **AUDITORIA**, capturando de forma fiel la fecha, hora, el usuario autenticado, el módulo impactado y la descripción exacta de los cambios efectuados.